

イラストで学ぶ音声認識 改訂第2版

10. End-to-End の音声認識

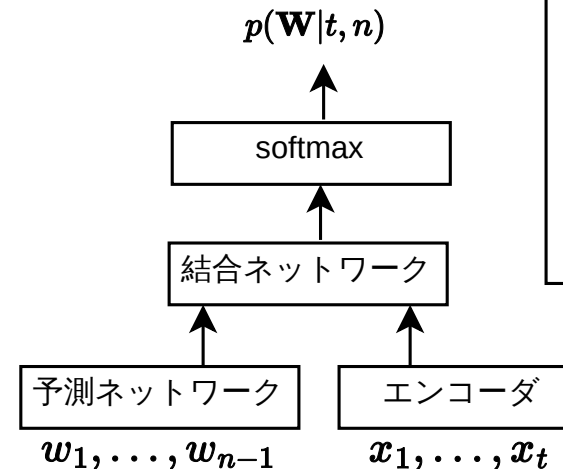
10.1 ディープニューラルネットワーク
による音声認識

10.2 CTC

10.3 seq2seq+アテンション

10.4 エンコーダの改良

10.5 RNN-トランスデューサ



このアーキテクチャで、音声入力に合わせて認識結果が出力されるストリーミング認識が可能になります。

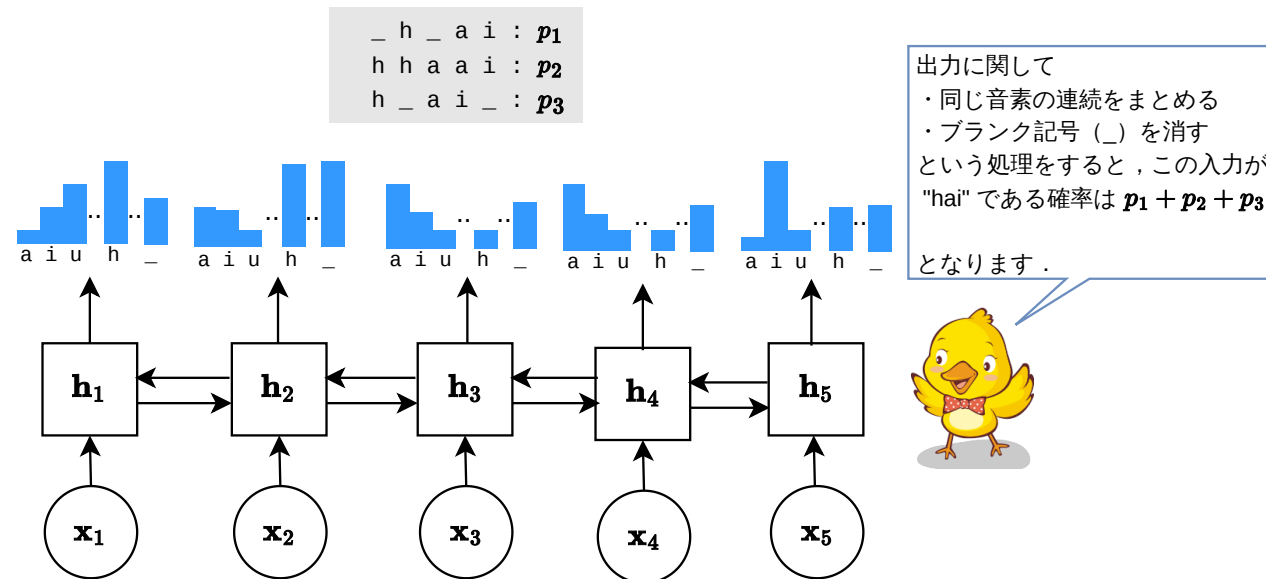


10.1 ディープニューラルネットワークによる音声認識

- 音声認識手法の変遷
 - 統計的音声認識
 - MFCC など，音響学の知見を活用した特徴抽出
 - 音響モデル(GMM-HMM)・言語モデル(N-gram)など個別に学習したモデルを組み合わせたものを WFST に変換して，ビームサーチで探索
 - ハイブリッド型音声認識
 - 音響モデルの学習に識別的な手法を導入 (DNN-HMM) したものと，ニューラルネットによる言語モデルを組み合わせる
 - 特徴抽出もDNNで行う
 - End-to-End音声認識
 - 入力と出力の対のみで一括学習する方式

10.2 CTC

- CTC (Connectionist Temporal Classification) とは
 - 異なる長さの系列 (例: 音声フレーム列と文字列) を対応付ける手法
 - フレームごとに音素またはブランクを出力し、すべてのアラインメント確率の和で出力系列の確率を算出



10.2 CTC

- CTCの学習
 - 出力系列の確率計算
 - $A(y)$ ：出力系列 y のアラインメント集合

$$P_{CTC}(y \mid x) = \sum_{A(y)} \prod_{t=1}^T P(a_t \mid x_t)$$

- HMM と同様に，フォワード・バックワードアルゴリズムを用いて効率的に計算可能
 - 損失関数： $-\log P(y \mid x)$
 - 音声信号と正解系列の対のみで学習が可能
 - 正解系列の確率を最大化しているのと同じ

10.2 CTC

- CTCの利点：単純な設計で大規模データでも高速な学習が可能
- CTCの欠点：出力が独立で文脈を考慮しづらい
- 言語モデルと組み合わせたデコーディング
 - α ：言語モデルの重み

$$P(y \mid x) = \log P_{CTC}(y \mid x) + \alpha \log P_{LM}(y)$$

- CTCが出力する記号系列を WFST デコーダへの入力とする方法もある

10.3 seq2seq+アテンション

- 単純な seq2seq モデルによる End-to-End 音声認識
 - エンコーダ・デコーダモデルで入出力の系列長の違いを吸収
 - 入力音声フレーム列) $X = x_1, x_2, \dots, x_T$ をエンコーダに入力し, 固定長のコンテキストベクトル $\mathbf{c}$ を生成

- 隠れ層ベクトル $\mathbf{h}_t$ の計算

$$\mathbf{h}_t = f_{enc}(\mathbf{h}_{t-1}, x_t)$$

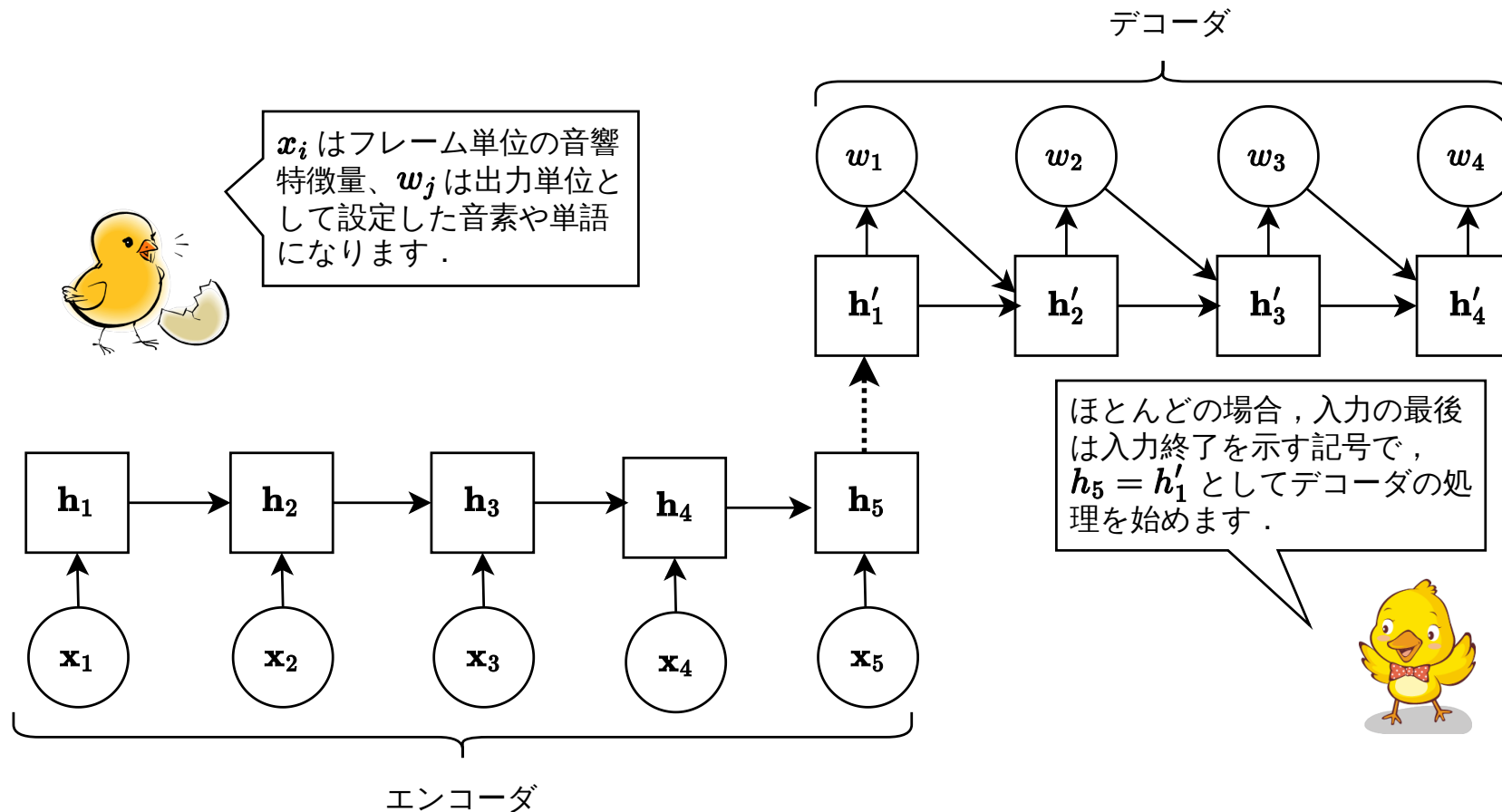
- コンテキストベクトル $\mathbf{c}$ からデコーダが逐次的に出力トークン列を生成
 - 時刻 u での隠れ層の計算 (ただし $\mathbf{h}'_0 = \mathbf{c}$)

$$\mathbf{h}'_u = f_{dec}(\mathbf{h}'_{u-1}, y_{u-1})$$

- 出力トークンの確率: $P(y_u \mid \mathbf{h}'_u) = \text{softmax}(\mathbf{W} \mathbf{h}'_u)$

10.3 seq2seq+アテンション

- 単純な seq2seq モデル



10.3 seq2seq+アテンション

- 単純な seq2seq モデルの学習
 - エンコーダ・デコーダ全体で「入力 X から正解ラベル列 $Y = y_1y_2 \dots y_U$ を再現する確率」を最大化
 - 実際は，交差エントロピー損失 $\mathcal{L}$ を最小化

$$\mathcal{L} = - \sum_{u=1}^U \ln P(y_u \mid y_{1:u-1}, X)$$

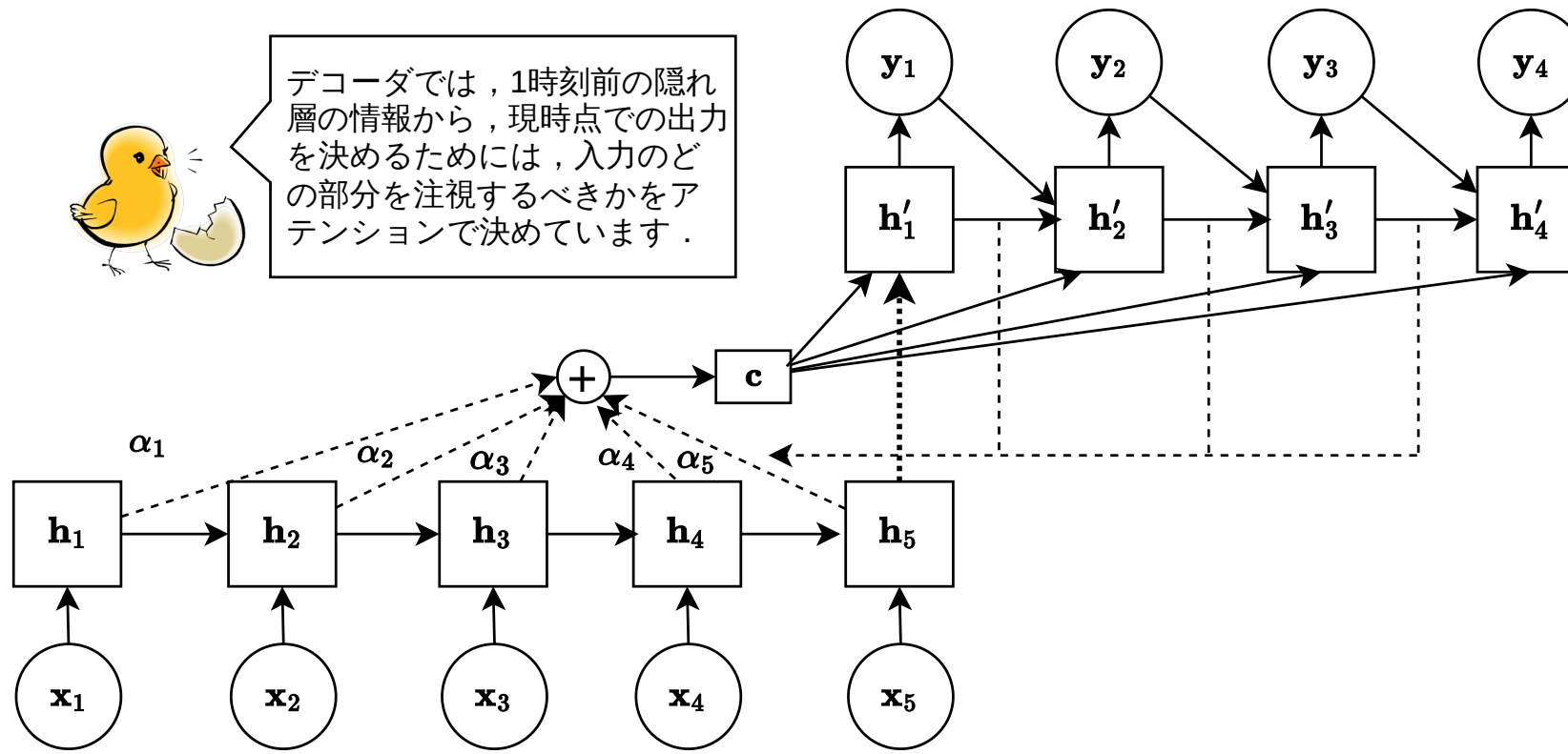
- 教師強制 (teacher forcing)
 - デコーダの学習中は前の時刻に出力した予測ラベルの代わりに正解ラベルを次の時刻へ入力する

10.3 seq2seq+アテンション

- アテンション機構の導入
 - 単純な seq2seq モデルでは、入力系列が長い場合に1つのベクトルに情報を集約することが困難
 - アテンション機構でデコーダが各タイミングで「どの入力フレームに注目するか」を学習することが可能
 - アテンションスコア: $e_t^u = f(\mathbf{h}_t, \mathbf{h}'_u)$
 - f は密結合ニューラルネットで学習
 - アテンション重み: $\alpha_t^u = \text{softmax}(e_t^u) = \frac{\exp(e_t^u)}{\sum_{t'=1}^T \exp(e_{t'}^u)}$
 - コンテキストベクトル: $\mathbf{c}_u = \sum_{t=1}^T \alpha_t^u \mathbf{h}_t$
 - デコーダの隠れ層の計算: $\mathbf{h}'_u = f_{dec}(\mathbf{h}'_{u-1}, y_{u-1}, \mathbf{c}_u)$

10.3 seq2seq+アテンション

- seq2seq モデルにアテンション機構を導入



10.4 エンコーダの改良

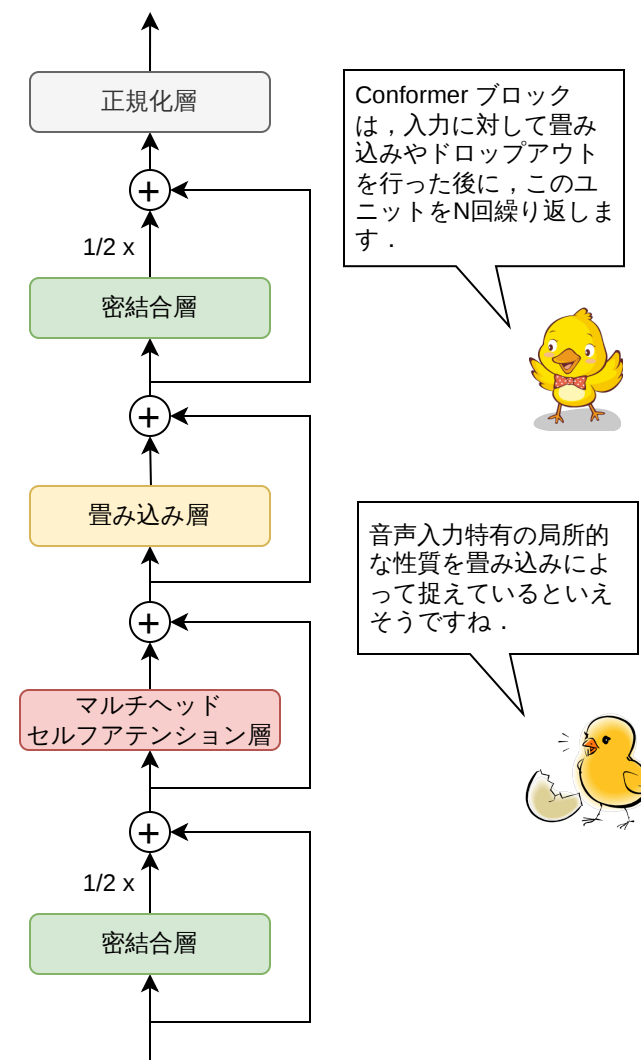
- エンコーダの進化
 - RNN/双方向RNN/LSTM：文脈保持を強化
 - Transformer：セルフアテンションでグローバルな文脈を捉える
 - Conformer/Branchformer：グローバル＋ローカルの特徴抽出
 - Conformer: 畳み込みとセルフアテンションを直列に組み合わせ
 - Branchformer: 並列処理で情報損失低減

10.4 エンコーダの改良

- Conformer
 - 入力された音響特徴系列は畳み込み層でサンプリングの処理が行われる
 - 後続処理の計算コストを削減するために、時間方向の解像度が低減される
 - その後、密結合層・ドロップアウト層を経て、 N 段の Conformer ブロックで特徴抽出が行われる
 - デコーダは単純な seq2seq + アテンションモデルを用いるだけで、十分な性能を発揮する

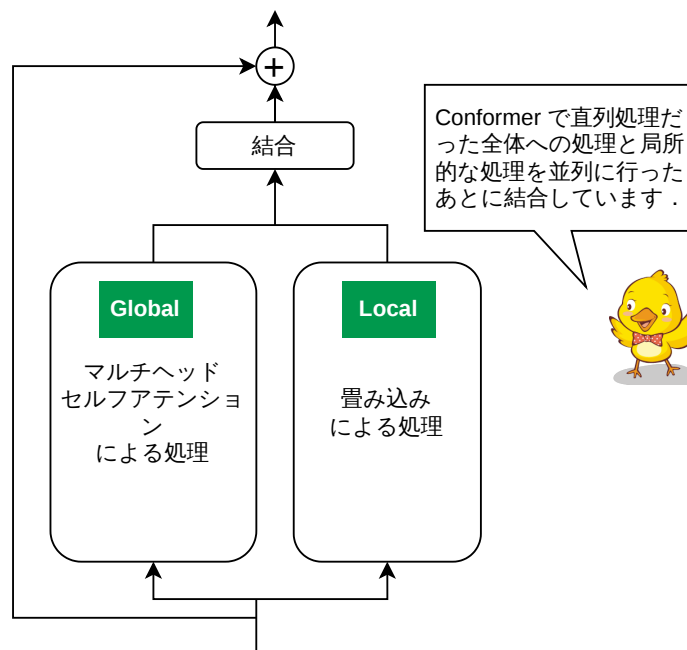
10.4 エンコーダの改良

- Conformer ブロックの構造
 - 前段密結合層：非線形変換による情報拡張
 - マルチヘッドセルフアテンション層：入力中の全ての位置間の相互作用を捉え，グローバルな依存関係を抽出する
 - 畳み込み層：局所的な特徴を抽出
 - 後段密結合層：さらに非線形変換を行う
 - 正規化層：出力を正規化し，学習を安定化



10.4 エンコーダの改良

- Branchformer
 - 多層ブロック内部においてセルフアテンションと畳み込みを並列に組み合わせることで、高速な処理が可能になり、情報の損失も低減
 - 単純な結合処理を畳み込みに置き換えたものが E-Branchformer

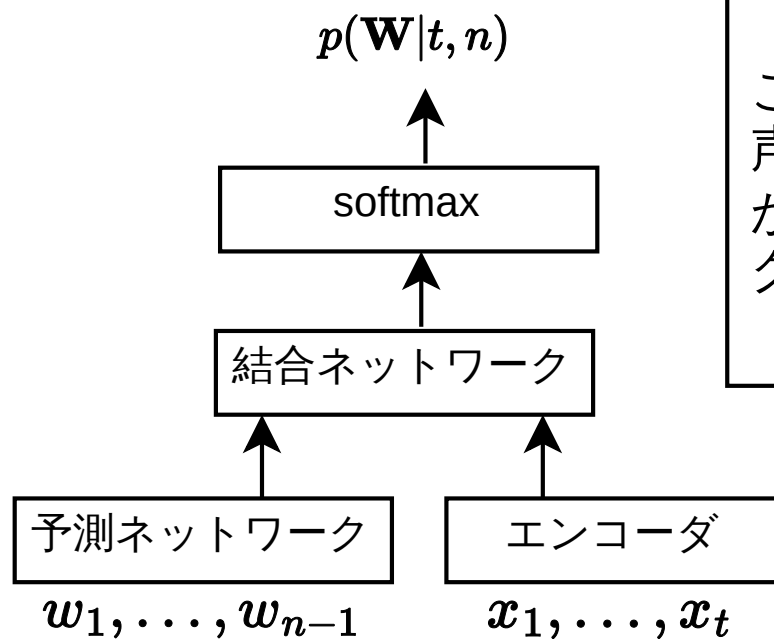


10.5 RNN-トランスデューサ

- CTCやseq2seqモデルは，音声入力全体が得られてから出力を生成することが前提
 - 字幕付与や対話システムなどではストリーミング音声認識と呼ばれるリアルタイム処理が必要
- RNN-トランスデューサ（RNN-T）
 - 音声入力を逐次的に処理しながら，出力を生成するモデル
 - エンコーダ：音声入力を時系列ベクトル化
 - 予測ネットワーク：出力ラベル履歴を保持
 - 結合ネットワーク：両者を統合して次ラベルを予測

10.5 RNN-トランスデューサ

- RNN-Tの構造



このアーキテクチャで、音声入力に合わせて認識結果が出力されるストリーミング認識が可能になります。



10.5 RNN-トランスデューサ

- エンコーダの処理

$$\mathbf{h}_t^{\text{enc}} = f_{\text{enc}}(\mathbf{h}_{t-1}^{\text{enc}}, x_t)$$

- 予測ネットワークの処理

$$\mathbf{h}_u^{\text{pred}} = f_{\text{pred}}(\mathbf{h}_{u-1}^{\text{pred}}, y_{u-1})$$

- 結合ネットワークの処理

$$z_{t,u} = f_{\text{joint}}(\mathbf{h}_t^{\text{enc}}, \mathbf{h}_u^{\text{pred}})$$

- 出力確率の計算 (w にはブランク記号も含む)

$$P(w \mid t, u) = \text{softmax}(\mathbf{W} z_{t,u})$$

10.5 RNN-トランスデューサ

- RNN-Tの学習
 - CTCと同様にフォワード・バックワードアルゴリズムを用いて損失値を計算
 - 入力フレーム軸 t と出力ラベル軸 u の2次元上で、「空白記号を出力して時間軸を進めるステップ」と「ラベルを出力してラベル列を進めるステップ」をすべて考慮した動的計画法
 - 損失関数： $\mathcal{L}_{\text{RNN-T}} = -\ln P(Y \mid X)$